VPC · 서브넷 · 라우팅 테이블 구성

VPC 생성

VPC 생성 정보

VPC는 AWS 클라우드의 격리된 부분으로서, Amazon EC2 인스턴스와 같은 AWS 객체로 채워집니다.

VPC 설정

생성할 리소스 정보

VPC 리소스 또는 VPC 및 기타 네트워킹 리소스만 생성합니다.

- (●) VPC만
- ( ) VPC 등

이름 태그 - *선택 사항*

'Name' 키와 사용자가 지정하는 값을 포함하는 태그를 생성합니다.

project-vpc

IPv4 CIDR 블록 정보

- (●) IPv4 CIDR 수동 입력
- ( ) IPAM 할당 IPv4 CIDR 블록

IPv4 CIDR

10.0.0.0/16

CIDR 블록 크기는 /16에서 /28 사이여야 합니다.

IPv6 CIDR 블록 정보

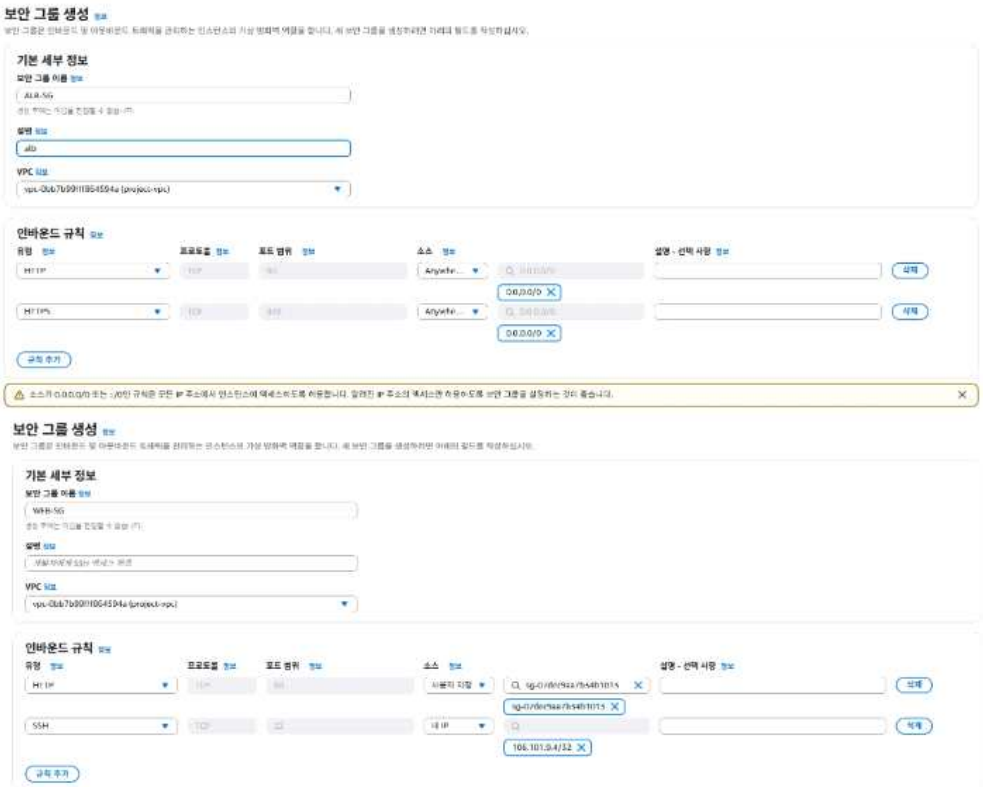
- (●) IPv6 CIDR 블록 없음
- ( ) IPAM 할당 IPv6 CIDR 블록
- ( ) Amazon 제공 IPv6 CIDR 블록
- ( ) 내가 소유한 IPv6 CIDR

테넌시 정보

기본값 ▼

보안 그룹 & UFW 방화벽 실습

1. 보안 그룹 3종 생성 (콘솔 경로: EC2 → 네트워크 & 보안 → 보안 그룹)



서브넷 생성 정보

VPC

VPC ID
이 VPC에 서브넷을 생성합니다.

vpc-0bb7b99fff864594a (project-vpc)

연결된 VPC CIDR

IPv4 CIDR
10.0.0.0/16

서브넷 설정
서브넷의 CIDR 블록 및 가용 영역을 지정합니다.

1/1개 서브넷

서브넷 이름
'Name' 키와 사용자가 지정하는 값을 포함하는 태그를 생성합니다.

my-subnet-01

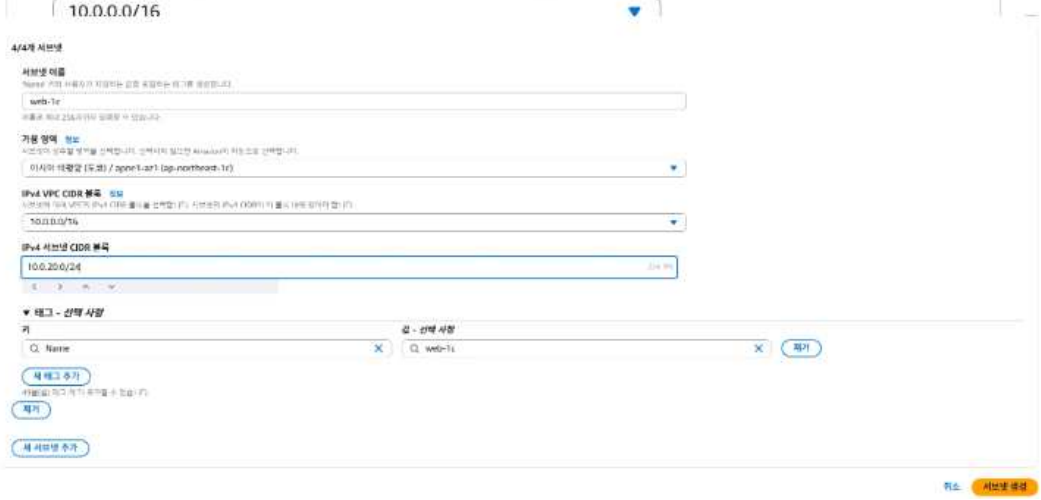
이름은 최대 256자까지 입력할 수 있습니다.

가용 영역 정보
서브넷이 상주할 영역을 선택합니다. 선택하지 않으면 Amazon이 자동으로 선택합니다.

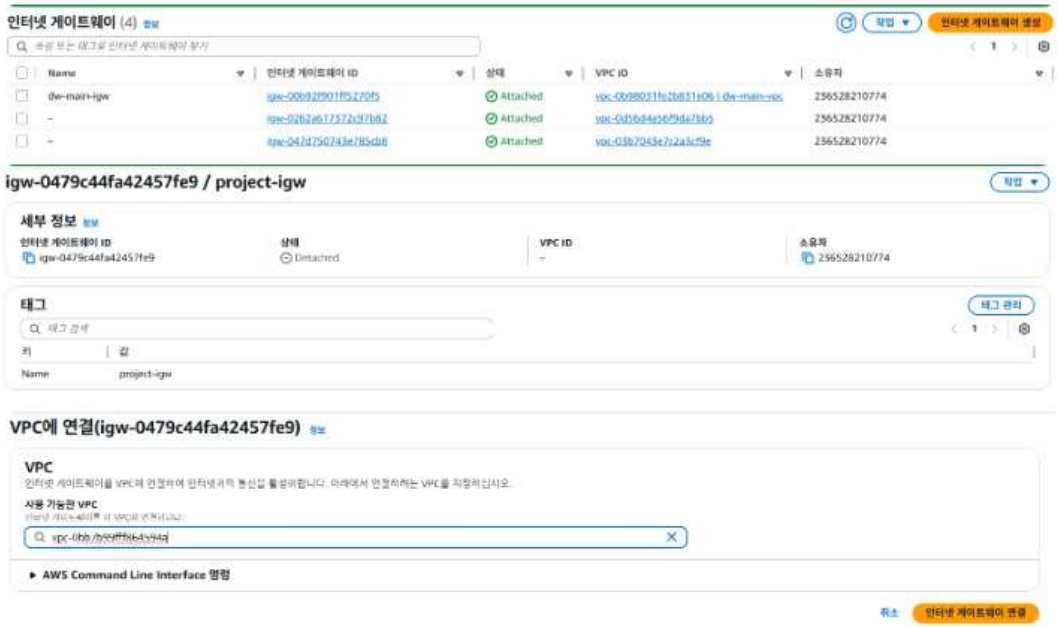
기본 설정 없음

IPv4 VPC CIDR 블록 정보
서브넷에 대해 VPC의 IPv4 CIDR 블록을 선택합니다. 서브넷의 IPv4 CIDR이 이 블록 내에 있어야 합니다.

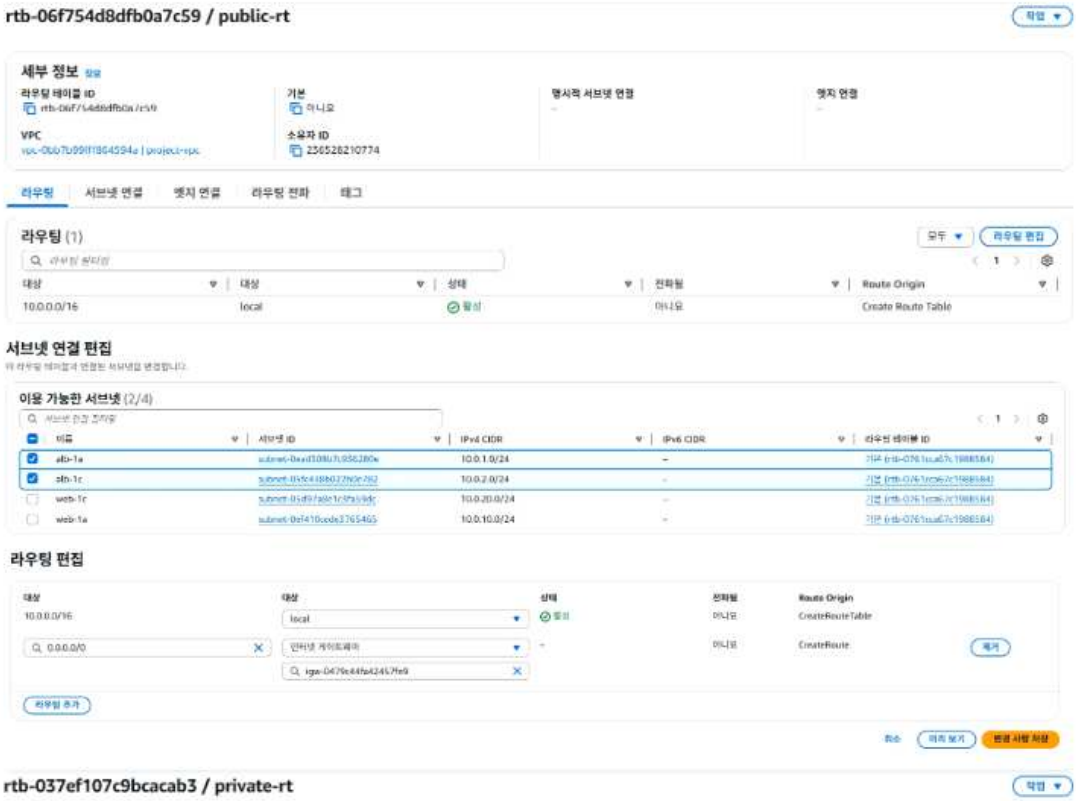
10.0.0.0/16



3) IGW(Internet Gateway) 생성 및 연결



4) 라우팅 테이블 구성

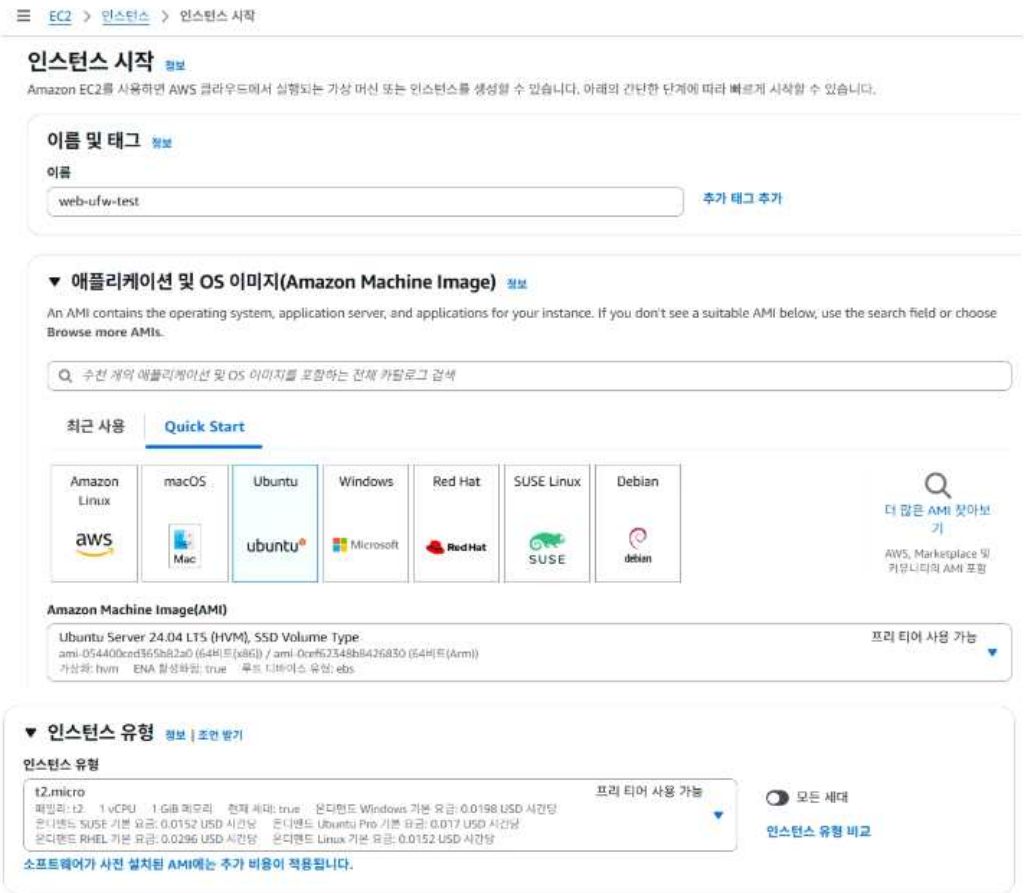




SSM Role 생성 (배스천 대신 Session Manager로 접속)



EC2 생성 (Web-SG 적용)



EC2 접속

```
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-10-0-10-217:~$
```

UFW 설치 및 초기 설정

이모저모 거친 과정이고 sudo ufw status 했을 때의 사진

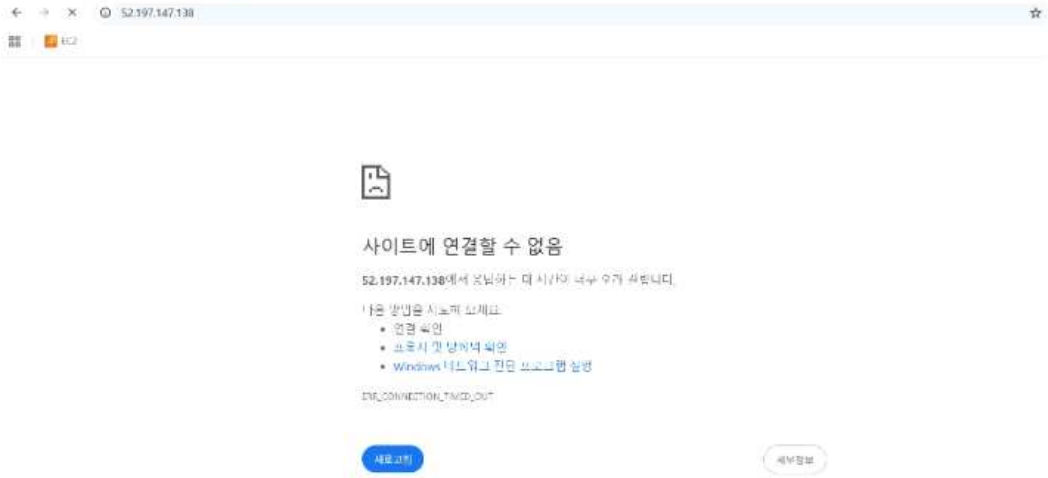
```
ubuntu@ip-10-0-10-217:~/flask-app$ sudo ufw status numbered
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] 22/tcp                     ALLOW IN    Anywhere
[ 2] 80/tcp                     ALLOW IN    Anywhere
[ 3] 443/tcp                    ALLOW IN    Anywhere
[ 4] Anywhere on docker0        ALLOW IN    Anywhere
[ 5] Anywhere                   ALLOW OUT   Anywhere on docker0        (out)
[ 6] 22/tcp (v6)                ALLOW IN    Anywhere (v6)
[ 7] 80/tcp (v6)                ALLOW IN    Anywhere (v6)
[ 8] 443/tcp (v6)               ALLOW IN    Anywhere (v6)
[ 9] Anywhere (v6) on docker0   ALLOW IN    Anywhere (v6)
[10] Anywhere (v6)              ALLOW OUT   Anywhere (v6) on docker0   (out)
```

방화벽 실습 포트 80,443,22번 허용했을 때 외부 사이트 접속 성공



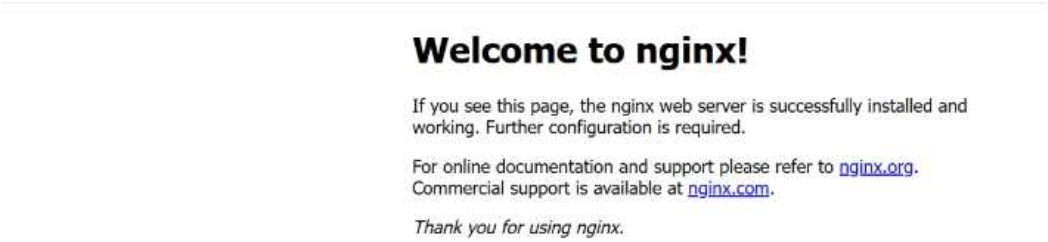
80번 포트 닫고 난 후 외부 접속 차단됨



내부에서 외부 IP로 접속 시도→아무것도 안 나옴

```
ubuntu@ip-10-0-10-217:~/flask-app$ curl http://52.197.147.138
```

재연결



앞으로 할 일

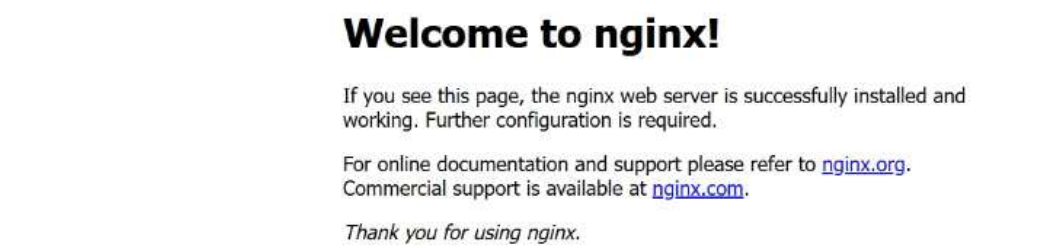
(EC2 User-Data로 서버 자동 구성하기->ALB + Auto Scaling Group 연결하기-> CloudWatch 모니터링 & 로그 수집 설정 ->GitHub Actions로 CI/CD 자동 배포 구현->부하 테스트로 Auto Scaling 동작 확인)

EC2 User-Data 기반 서버 자동 구성

User-Data 스크립트 작성(우분투용)

스크립트

13.114.44.33



Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

EC2 실행 후 확인

컨테이너 상태 확인

```
ubuntu@ip-10-0-10-217:~$ docker ps
CONTAINER ID   IMAGE                COMMAND           CREATED       STATUS
    PORTS     NAMES
2fa2e3cd5fe5   flask-image:latest   "python app.py"   2 hours ago   Up 57 second
s             flask-app
```